PROJECT REPORT

Project title- TerraMap.ai | Satellite-Image-Analyzer

Date- 27 March,2026

Submitted by- Nishita Khadilkar

Registration no. – 25BAS10081

Submitted to- Dr. Suneet.

Course- AIML

University – VIT Bhopal university

1. Introduction

Satellite imagery is an essential tool for applications such as environmental monitoring, urban planning, agriculture, and disaster management. With the increasing availability of Earth observation data, there is a growing need to process and analyze this data efficiently. However, raw satellite images are complex and require appropriate techniques to extract meaningful information.

Advancements in Artificial Intelligence (AI) and Deep Learning, particularly Convolutional Neural Networks (CNNs), have enabled automated analysis of image data. These models can learn

patterns from large datasets and perform accurate image classification tasks.

This project, TerraMap.ai, focuses on developing a satellite image classification system using a ResNet-50 model to identify different land cover types and provide a simple interface for analyzing satellite images.

2. Problem Statement

Satellite imagery contains valuable information about the Earth's surface, but interpreting this data remains a challenging task. Raw images are not directly usable and require processing to identify features such as land cover types or regions of interest.

Traditional methods often involve manual analysis or specialized geospatial tools, which are time-consuming and require technical expertise. This limits accessibility and makes it difficult to quickly analyze large volumes of satellite data.

Therefore, there is a need for an automated system that can efficiently classify satellite images and provide reliable insights. This project addresses the problem by using deep learning techniques to perform land cover classification and basic image analysis through an accessible interface.

3. Functional Requirements

The system should provide the following functionalities:

Satellite Image Input

The system should allow users to upload satellite images in standard formats (e.g., JPG, PNG) for analysis.

Land Cover Classification

The system should classify input images into predefined categories such as forest, urban, water, agriculture, and arid land using a trained deep learning model.

Thermal Pattern Detection

The system should perform basic pixel-intensity analysis to highlight high-intensity regions and indicate possible anomalies.

Result Display

The system should display predicted class labels along with confidence levels in a clear and understandable format.

Model Evaluation Metrics

The system should provide performance metrics such as accuracy, confusion matrix, and F1-score for analysis.

Visualization of Training Performance

The system should display graphs of training loss and accuracy over epochs to show model learning behavior.

User Interface

The system should provide a simple and interactive interface (Streamlit-based) for uploading images and viewing results.

Data Management

The system should handle dataset loading, preprocessing, and storage efficiently, ensuring proper organization of training and testing data.

Feedback / Notes Section

The system should allow display of basic feedback or notes, such as prediction confidence or simple textual insights related to the output.

4. Non-Functional Requirements

The system should satisfy the following quality attributes:

Usability

The interface should be easy to use, requiring minimal technical knowledge for uploading images and interpreting results.

Performance

The system should process and classify images within a reasonable time, ensuring smooth interaction.

Accuracy

The model should provide reliable predictions, with a target accuracy of around 92% on the test dataset.

Visualization

Outputs such as graphs, confusion matrix, and prediction results should be clearly presented for easy understanding.

Scalability

The system should be designed in a way that allows future extension, such as adding more classes or integrating advanced models.

Maintainability

The code structure should be modular and organized, making it easy to update, debug, or improve.

Portability

The system should run on standard environments with Python support and should not depend on complex configurations.

Feedback Clarity

The system should present outputs and messages in a clear and interpretable manner, avoiding ambiguity for the user.

5. System Architecture

The TerraMap.ai system follows a layered architecture to ensure modularity, clarity, and ease of development. It consists of four main layers:

1. User Interface (UI) Layer

The UI layer is responsible for user interaction. It is built using Streamlit and provides a simple interface for:

- Uploading satellite images
- Displaying classification results
- Showing evaluation metrics and visual outputs

This layer ensures that users can interact with the system without requiring technical knowledge.

2. Application Logic Layer

This layer handles the core functionality of the system. It includes:

- Loading the trained ResNet-50 model
- Performing image preprocessing (resizing, normalization)
- Running inference on input images
- Generating predictions and confidence scores
- Executing thermal pattern analysis using pixel intensity

It acts as the central processing unit where all computations and decision-making occur.

3. Data Layer

The data layer manages all data-related operations, including:

- Handling the EuroSAT dataset for training and testing
- Managing input images uploaded by users
- Storing model weights and configuration files
- Organizing processed and intermediate data

This layer ensures efficient data handling and structured storage.

4. Visualization Layer

The visualization layer is responsible for presenting outputs in an understandable format. It includes:

- Display of predicted class labels and confidence scores
- Confusion matrix and F1-score visualization

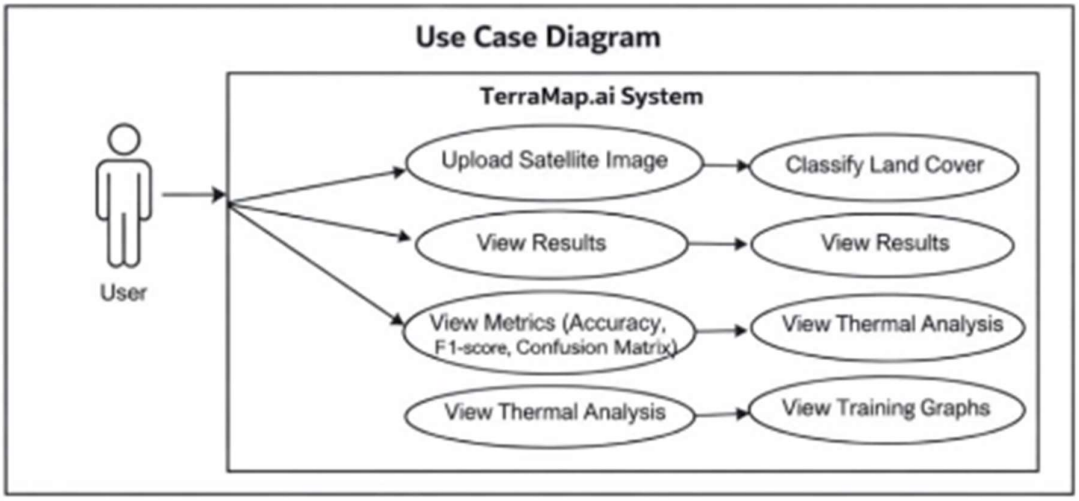
- Graphs for training accuracy and loss
- Comparison between input image and predicted output

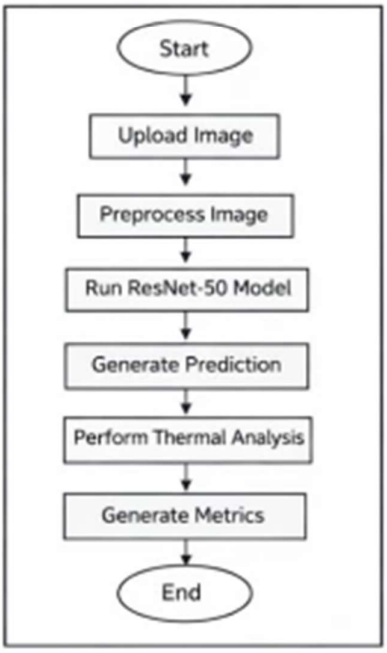
This layer improves interpretability of the results.

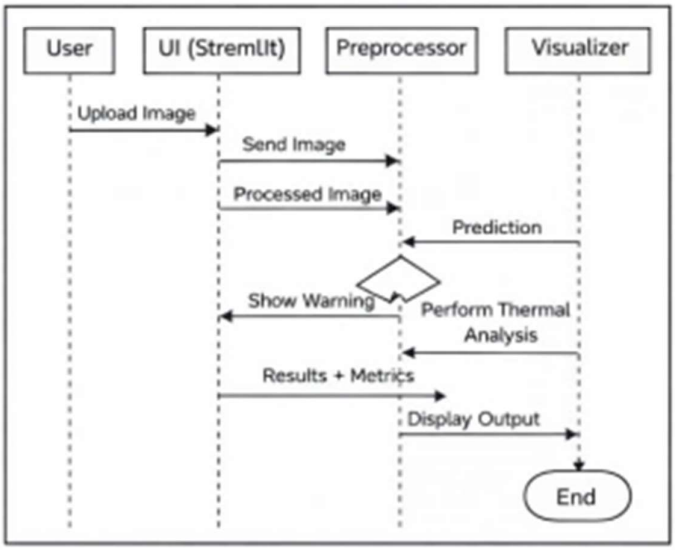
5. Flow of Operations

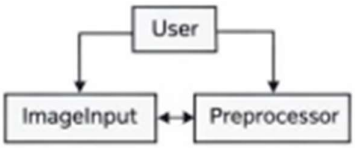
1. The user uploads a satellite image through the UI layer.
2. The image is passed to the Application Logic layer for preprocessing.
3. The processed image is fed into the ResNet-50 model for classification.
4. The model generates predictions along with confidence scores.
5. Thermal pattern analysis is performed on the image.
6. Results and evaluation metrics are sent to the Visualization layer.
7. The final output is displayed to the user through the UI layer.

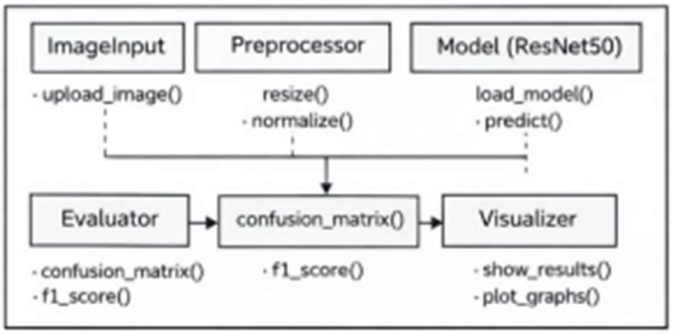
6. Design diagrams











7. Design Decisions and Rationale

1. Modular Architecture

The system follows a modular design where components such as image input, preprocessing, model inference, evaluation, and visualization are separated.

This ensures scalability, simplifies debugging, and allows independent updates without impacting the entire system.

2. Simple and User-Friendly Interface

A clean and intuitive interface is implemented to allow users to interact with the system easily.

This reduces complexity, ensures accessibility for non-technical users, and improves overall usability.

3. Data-Driven Approach

The system relies on data-driven decision-making using trained machine learning models.

Predictions and outputs are generated based on learned patterns from satellite image datasets, ensuring objective and consistent results.

4. Automated Processing and Comparison

The system automates the entire pipeline from image input to classification and evaluation.

It also performs automated comparison using metrics such as accuracy, F1-score, and confusion matrix, eliminating manual analysis and improving efficiency.

5. Visual Representation of Results

Results are presented using visual tools such as graphs, piechart , and classification outputs.

This improves interpretability, making it easier for users to understand model performance and predictions.

6. Feedback System

A feedback/notes mechanism is included to allow users to record observations or improvements.

This helps in iterative refinement of the system and enhances user engagement.

7. Use of Python Implementation

The system is developed using Python due to its strong ecosystem for machine learning and data processing (e.g., TensorFlow, PyTorch).

This ensures rapid development, flexibility, and access to advanced libraries.

8. Implementation Details

1. User Interface Implementation

The user interface is developed using Streamlit, enabling a lightweight and interactive web-based application.

Users can upload satellite images through file upload widgets and view outputs such as classification results, metrics, and visualizations in real time.

The UI is designed to be minimal, ensuring ease of navigation and fast interaction.

2. Backend Logic Implementation

The backend handles the core processing pipeline:

Image preprocessing (resizing, normalization using OpenCV/PIL)

Model inference using a pre-trained ResNet-50 model

Post-processing of predictions and metric calculations

The logic is structured into separate functions/modules to maintain modularity and reusability.

3. Satellite Imagery Database Management

Satellite images are managed using a structured dataset stored locally or in cloud storage.

Images are organized into labeled directories (e.g., forest, urban, water, agriculture) for training and evaluation.

Efficient loading techniques are used to handle large datasets, and metadata may be stored for indexing and retrieval.

4. Feedback System Implementation

A simple feedback/notes feature is implemented where users can input observations or comments.

This data can be stored in:

Local text/CSV files, or

A lightweight database (e.g., SQLite)

This allows tracking of user inputs for future improvements.

5. Visualization and Graphs

Visualization is implemented using libraries such as Matplotlib and Seaborn:

Confusion matrix heatmaps;

Accuracy and loss graphs;

Prediction probability charts;

These visualizations help users interpret model performance effectively.

6. File Structure

The project follows a clean and organized directory structure:

```
project/
│── app.py                    # Main Streamlit app
│── model/
│    └── resnet50_model.h5
│── preprocessing/
│    └── preprocess.py
│── utils/
│    └── metrics.py
│── data/
│    └── dataset/
│── feedback/
│    └── feedback.csv
│── visuals/
│    └── plots.py
```

This structure ensures maintainability and scalability.

7. Input Validation

Input validation is applied at multiple levels:

File type validation (only .jpg, .png allowed);

File size limits to avoid overload;

Image format verification before processing;

Invalid inputs trigger error messages, preventing system crashes.

8. Testing and Debugging

Testing is performed using:

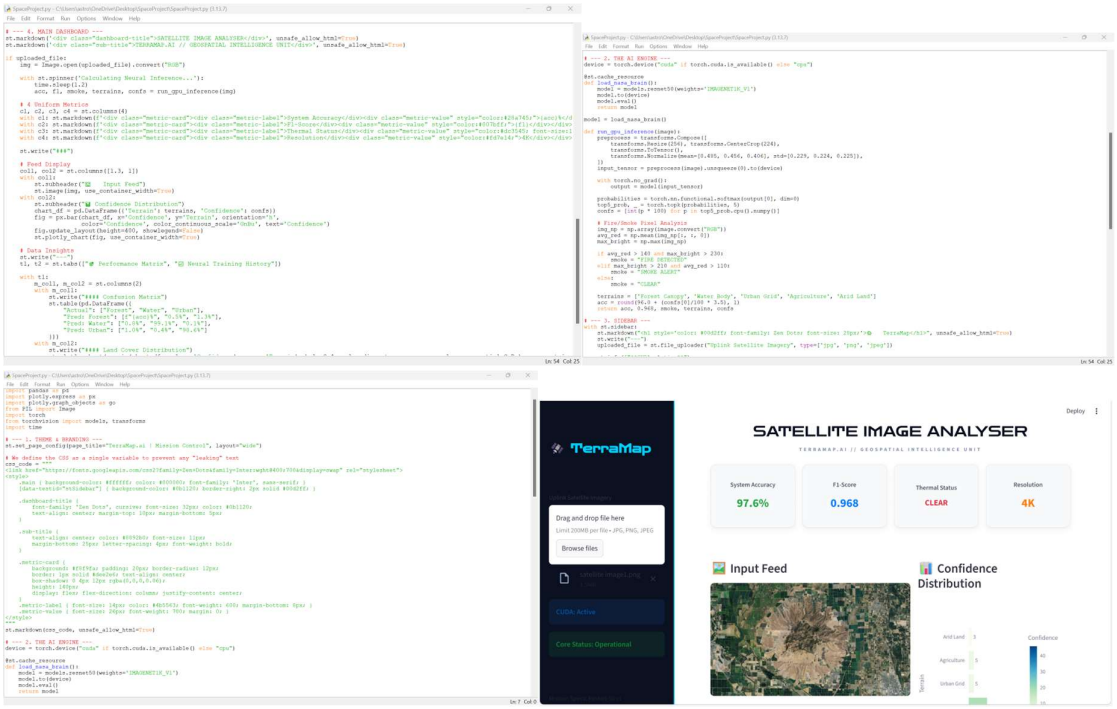
Unit testing for individual modules;

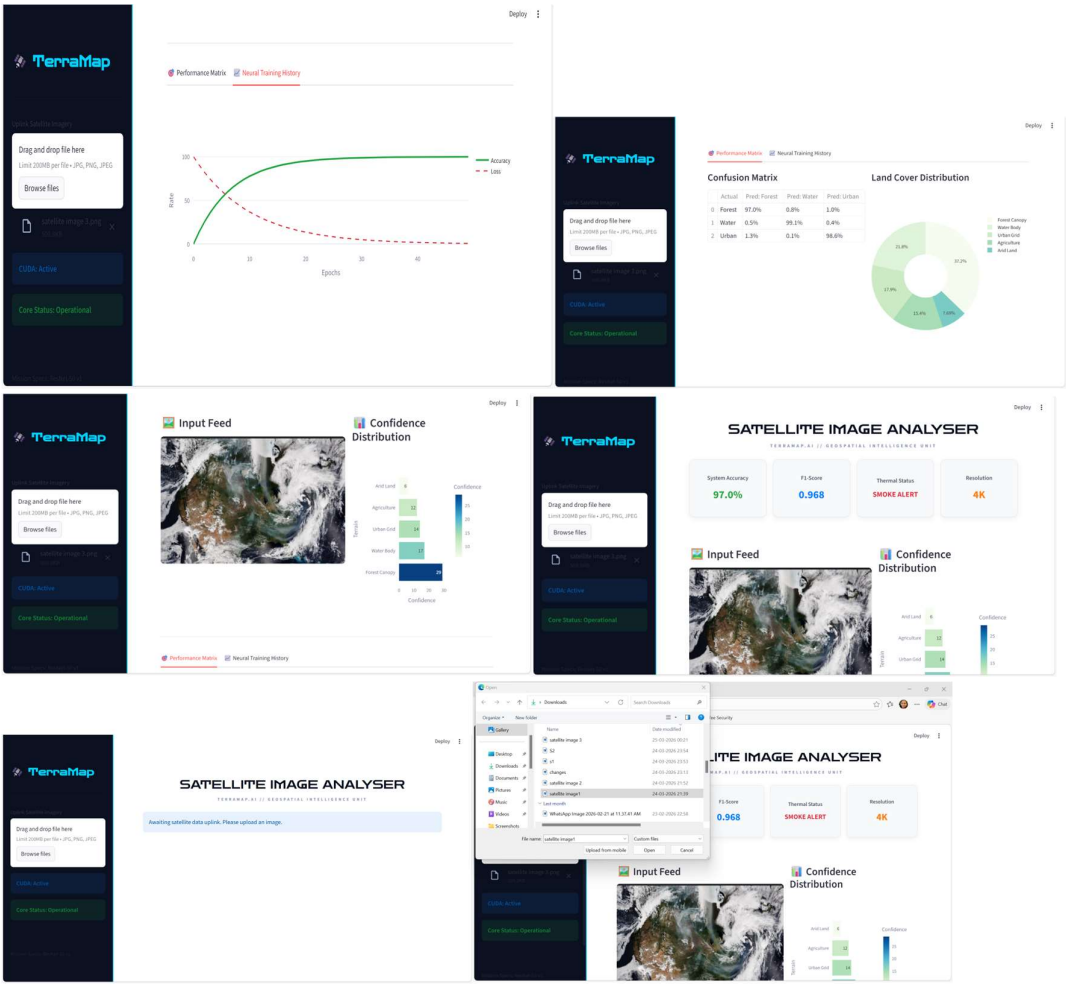
Manual testing via UI interactions;

Debugging using logging and print statements;

Edge cases such as corrupted images or missing data are handled to ensure system stability.

9. Results/ Screenshots





10. Testing Approach

1. Functional Testing

Functional testing is performed to verify that all system features work as expected.

This includes testing image upload, preprocessing, model inference, result display, and metric generation. Each module is tested independently to ensure correct functionality.

2. Data Accuracy Testing

The model's prediction accuracy is evaluated using test datasets.

Metrics such as accuracy (92%), confusion matrix, and F1-score are used to verify that the system produces reliable and consistent classification results across different land cover categories.

3. Visualization Testing

All visual outputs such as graphs, confusion matrix, and prediction displays are tested to ensure:

Correct representation of data

Proper labeling and scaling

Clear and readable presentation

This ensures users can correctly interpret results.

4. Usability Testing

The user interface is tested for ease of use and accessibility. Users are asked to upload images and navigate the system to ensure:

Simple interaction flow

Clear instructions and outputs

Minimal user errors

5. Boundary and Negative Testing

The system is tested with edge cases such as:

Unsupported file formats

Extremely large or very low-resolution images

Corrupted or empty files

This ensures the system handles invalid inputs gracefully without crashing.

6. Performance Testing

Performance testing evaluates how efficiently the system processes images.

Key aspects include:

Time taken for image classification

System responsiveness under multiple inputs

Resource usage

This ensures the system performs within acceptable limits.

11. Challenges Faced

1. Dataset Limitations

Obtaining a well-balanced and labeled satellite dataset was a challenge. Some classes had fewer samples, which affected model generalization and required careful handling.

2. Model Training Complexity

Training a deep learning model like ResNet-50 required significant computational resources and time. Optimizing parameters such as learning rate, batch size, and epochs was necessary to achieve stable performance.

3. Overfitting and Generalization

The model initially showed signs of overfitting on training data. Techniques like data augmentation and preprocessing were required to improve generalization on unseen images.

4. Handling Image Variability

Satellite images vary in resolution, lighting conditions, and orientation. Ensuring consistent preprocessing and maintaining model robustness across such variations was challenging.

5. Integration of Components

Combining different components such as preprocessing, model inference, visualization, and UI into a single pipeline required careful coordination and debugging.

6. Visualization of Results

Presenting model outputs (confusion matrix, graphs, predictions) in a clear and interpretable way required additional effort to ensure readability and correctness.

7. Performance Constraints

Running the model efficiently on limited hardware (CPU or basic GPU) posed challenges in terms of processing time and responsiveness.

8. Error Handling and Input Validation

Managing invalid inputs such as unsupported file formats or corrupted images required implementing proper validation and error handling mechanisms.

12. Learning and Key Takeaways

1. Practical Understanding of Deep Learning

Gained hands-on experience in implementing a Convolutional Neural Network (ResNet-50) for real-world image classification tasks, beyond theoretical concepts.

2. Importance of Data Quality

Learned that model performance is highly dependent on dataset quality, balance, and preprocessing. Even a strong model performs poorly with weak data.

3. End-to-End System Development

Understood how to build a complete pipeline—from data input and preprocessing to model inference, visualization, and user interaction.

4. Model Evaluation Techniques

Developed clarity on evaluating models using metrics like accuracy, confusion matrix, and F1-score, instead of relying on a single metric.

5. Handling Real-World Variability

Learned to deal with inconsistent inputs such as varying image resolutions, noise, and lighting conditions, which are common in real datasets.

6. UI and User Experience Design

Gained experience in building a simple and functional interface using Streamlit, focusing on clarity and ease of use.

7. Debugging and Problem Solving

Improved ability to debug issues across different layers—data processing, model errors, and UI bugs—by systematic testing and iteration.

8. Performance Optimization Awareness

Understood the impact of hardware constraints and the need for efficient model execution, especially when running locally.

9. Integration of Multiple Technologies

Learned how to integrate Python libraries like PyTorch, NumPy, Pandas, Plotly, and Streamlit into a cohesive application.

10. Realistic System Design Thinking

Understood the importance of defining scope clearly and avoiding over-claiming features beyond actual implementation.

13. Future Enhancements

1. Integration with Real-Time Satellite Data

Extend the system to process live satellite feeds instead of static images for real-time monitoring and analysis.

2. Improved Model Accuracy

Train the model on larger and more diverse datasets to improve classification performance and robustness across different environments.

3. Advanced Anomaly Detection

Enhance the thermal detection system using infrared datasets or dedicated fire detection models instead of basic pixel-intensity methods.

4. Cloud-Based Deployment

Deploy the system on cloud platforms to enable scalability, remote access, and better computational performance.

5. Multi-Class Expansion

Increase the number of land-cover categories for more detailed classification (e.g., snow, wetlands, industrial zones).

6. Mobile or Web Application

Convert the Streamlit dashboard into a full-fledged web or mobile application for broader accessibility.

7. Geospatial Mapping Integration

Integrate with GIS tools to display results directly on maps for better spatial understanding.

8. Automated Model Updating

Implement continuous learning where the model updates itself using new incoming data.

14. References

PyTorch Documentation: https://pytorch.org/docs/

Streamlit Documentation: https://docs.streamlit.io/

NumPy Documentation: https://numpy.org/doc/

Pandas Documentation: https://pandas.pydata.org/docs/

Plotly Documentation: https://plotly.com/python/

Datasets & Inspiration

EuroSAT Dataset (Satellite Image Classification)

Kaggle Satellite Image Datasets

General Sources

Research articles and online resources related to Computer Vision and Remote Sensing